Effect of Tool-Count Context on MCP Tool Selection

MCPToolBench++ subset, 5 vs 20 vs 50 tools in context. AI465 Agentic AI, SVNIT Surat, Semester VII. Research Project Phase 2 report. Roll no. U23AI052 (individual work).

Status: experiment not yet run. This version contains the objective, deviations, implementation and setup. Sections 4 and 5 are filled automatically from the saved traces once the run is done: `python scripts/make_report.py --run-dir results/<run>`. No number in this document comes from a model run.

1. Objective and declared deviations

Approved objective (Phase 1). Evaluate the effect of tool-count context on MCP tool-selection accuracy and failure composition for single-step and multi-step tool-calling tasks on a 200-task stratified subset of MCPToolBench++ (search, map, finance), at a matched tool-count budget (5 vs 20 vs 50 tools in context), measured by AST accuracy, AST DAG accuracy, tokens per task, model calls per task and wall-clock latency per task. It targets three gaps in the paper (Fan et al., 2025, arXiv:2508.07575): no cost or latency, no variance, no failure taxonomy.

#	Deviation from the approved objective	Reason
D1	Subset is 200 **single-step** tasks (67 search, 67 map, 66 finance); no multi-step tasks, so AST DAG accuracy is not reported.	The public repository (commit 2bd35e5) ships single-step data for these domains; map has only 2 multi-step items out of 500. Scope reduction, reported as allowed.
D2	No execution-based Pass@K.	It needs live MCP servers with third-party credentials. It was not in the approved metric list.
D3	The benchmark's GPT-4o judge is replaced by a deterministic scorer (`rp2/scoring.py`).	Removes judge/model circularity and cost. Checked by an oracle test (the 200 gold labels score 100%), the unit-test suite, and agreement with the author's own labels on 20 tasks is measured by `scripts/label_audit.py` (not yet done).
D4	20- and 50-tool contexts add **distractors from a pool of 87 tools** taken from all six benchmark categories.	A task's native list is 1 (finance), 5 (search) or 32 (map) tools. Gold tool plus at most 2 labelled similar tools are always present; contexts are nested (5 is a subset of 20 is a subset of 50).
D5	"Matched budget" is implemented as (a) equal prompts across conditions and (b) a 5-tool baseline given k=8 samples with majority vote so its token spend is close to the 50-tool condition.	Rubric item 2.4: a method that costs more must be compared against a baseline given the same budget.
D6	A missing required parameter counts as a parameter error.	Stricter than the benchmark judge, which only checks parameters that were predicted.

2. What was implemented

- **Data** (`scripts/build_subset.py`): seeded stratified sample from the benchmark clone, saved to `data/subset_200.json` with the 87-tool pool; the checkout is self-contained.
- **Condition builder** (`rp2/context.py`): the only thing that differs between conditions is the tool list. One seeded permutation of the pool per task gives the nested design; order is a seeded shuffle.
- **Prompts** are files (`prompts/`), identical across conditions; a test asserts this. The padding arm appends neutral text of exactly the length difference between that task's 50- and 5-tool contexts.
- **Runner** (`rp2/runner.py`): provider adapters for Anthropic and OpenAI-compatible endpoints, retries with backoff, global rate limit, resume, jobs interleaved across conditions within each run so model drift hits every condition equally. A failed call becomes an `api_error` record, never a crash.
- **Scorer and taxonomy**: tool_ok (gold or a labelled similar tool), param_ok (gold values compared with numeric, enum, lat/lon (0.1) and token-F1 ≥ 0.5 rules, as in the judge prompt), and nine mutually exclusive failure labels.
- **Analysis**: mean ± SD over runs, paired bootstrap over tasks (10,000 resamples), tokens per correct task, domain and position breakdowns, and an automatically selected traced failure case.
- **Ablations** (each isolates one factor): position of the gold tool (first / last / shuffled at 50 tools); context length (5 tools padded to the 50-tool length); budget (5 tools with self-consistency).
- **Own vs borrowed.** About 2,151 lines of Python written for this project (runner, conditions, scorer, statistics, report, slides, tests). Borrowed: the benchmark's task data and labels (github.com/mcp-tool-bench/mcptoolbenchpp, commit 2bd35e5) and the parameter-matching rules from its judge prompt, re-implemented. No benchmark code is imported.

3. Experimental setup (pinned)

Model requested	to be set in configs/config.yaml (exact version string)	Model string returned by API	recorded per call in the traces
Provider	anthropic or openai_compat (free-tier or local endpoint)	Dates of runs	recorded in manifest.json
Temperature / top-p	0.7 / 1.0	Seed	base 1000 + run*100000 + task index, where the provider supports seeds
Runs per condition	3 full passes over 200 tasks	max_tokens	512

Task metric: AST accuracy (tool and parameters correct) and tool-selection accuracy. Cost: model calls, prompt and completion tokens (provider-reported) and wall-clock latency per task. Every run writes a full trace (prompts, tool list, raw response, usage, score) to `traces/`; `python scripts/aggregate.py` regenerates every table from them. Planned cost (chars/4 estimate, input tokens): about 8.2M for the three core conditions and 29.8M with all ablations over 3 runs.

4. Results

Pending. Results table, accuracy plot, cost plot and failure-composition plot are generated by `scripts/aggregate.py` and inserted here by `scripts/make_report.py`.

5. Analysis

Pending. Findings (effect sizes with bootstrap intervals, cost, failure composition, ablations, matched budget) are computed from the saved results; at least one failure case is read from its trace and shown here.

6. Limitations and four more weeks

- One model and one temperature: conclusions are about this model on this date, not about tool count in general. Single-step tasks only; no Pass@K or live MCP execution.
- Labels are the benchmark's. A "wrong tool in the same domain" may be a valid alternative that the labelled `similar_tools` list misses, which inflates same-domain errors; the traced case and the 20-task audit are the guard.
- Distractors come from other domains and are mostly easy to dismiss, so the 50-tool condition is gentler than a marketplace of near-duplicate tools. Parameter matching is lenient (containment and token F1), as in the benchmark. The public benchmark may be in the model's training data.
- **With four more weeks:** several models and sizes; distractors chosen by embedding similarity to the gold tool; a retrieval baseline that shows the top-k tools; multi-step chains built by composing the labelled calls; live MCP execution for Pass@K; failure labels audited by a second person.